

SCP, RSYNC & SFTP

Command Reference, Real-World Use Cases & Comparison Guide

This guide covers the most commonly used commands and flags for SCP, RSYNC, and SFTP — three of the core tools used for secure file transfer over SSH in Linux/Unix system administration, DevOps, and homelab environments. Each command includes what it does and a practical scenario where it's actually used, followed by a head-to-head comparison of when to reach for each tool.

1. SCP (Secure Copy Protocol)

SCP copies files/directories between hosts over SSH. It is simple, fast for one-off transfers, and available on almost every Linux system by default. It has no resume or sync capability — every run is a fresh, full copy.

Core Commands

```
scp file.txt user@remotehost:/home/user/
```

What it does: Copies a single local file to a remote server's specified directory.

Real-world use case: Pushing a config file or a build artifact to a remote server during deployment.

```
scp user@remotehost:/home/user/file.txt .
```

What it does: Downloads a single file from a remote server to the current local directory.

Real-world use case: Pulling a log file from a production server to inspect it locally.

```
scp -r ./project user@remotehost:/var/www/
```

What it does: Recursively copies an entire directory (and its contents) to the remote host.

Real-world use case: Deploying a whole website/project folder to a web server in one shot.

```
scp -P 2222 file.txt user@remotehost:/home/user/
```

What it does: Uses a non-default SSH port (-P, capital) for the transfer.

Real-world use case: Connecting to a server that has SSH exposed on a hardened, non-standard port.

```
scp -i ~/.ssh/id_rsa_key file.txt user@remotehost:/tmp/
```

What it does: Uses a specific private key file for authentication instead of the default key.

Real-world use case: Transferring files to a cloud VM (AWS EC2, Azure) that requires a specific .pem key.

```
scp -C largefile.iso user@remotehost:/data/
```

What it does: Enables compression (-C) during transfer to speed up copying over slow links.

Real-world use case: Sending a large ISO or backup archive over a limited-bandwidth WAN link.

```
scp -p script.sh user@remotehost:/opt/scripts/
```

What it does: Preserves modification times, access times, and file permissions (-p, lowercase) of the source file.

Real-world use case: Copying scripts where the original timestamps must be retained for audit/versioning.

```
scp -l 8000 bigfile.tar.gz user@remotehost:/backup/
```

What it does: Limits transfer bandwidth to the given Kbit/s value.

Real-world use case: Throttling a large backup copy so it doesn't saturate the network during business hours.

```
scp file1.txt file2.txt user@remotehost:/home/user/
```

What it does: Copies multiple files to the same remote destination in a single command.

Real-world use case: Pushing several related config files (nginx.conf, ssl.conf) together.

```
scp user1@host1:/data/file.txt user2@host2:/data/
```

What it does: Copies a file directly between two remote hosts (traffic still routes through the local machine by default, or directly with -3 in some setups).

Real-world use case: Migrating a file from an old server to a new server without downloading it locally first.

```
scp -v file.txt user@remotehost:/home/user/
```

What it does: Verbose mode — prints detailed debug/connection information during transfer.

Real-world use case: Troubleshooting why an SCP transfer is failing or hanging (auth, permissions, network).

When SCP shines

- Quick one-time copy of a single file or small folder to/from a remote server.
- Scripting a simple push/pull step inside a CI/CD or shell automation task.
- Environments where rsync isn't installed but SSH/SCP is guaranteed to exist.

2. RSYNC (Remote Sync)

rsync synchronizes files/directories, transferring only the differences (delta) between source and destination. It supports resuming interrupted transfers, preserving permissions/ownership, deleting stale files on the destination, and works both locally and over SSH. This makes it the tool of choice for backups and large repeated syncs.

Core Commands

```
rsync -avz ./project/ user@remotehost:/var/www/project/
```

What it does: The most common rsync combo: -a (archive: preserves permissions, timestamps, symlinks, ownership recursively), -v (verbose), -z (compress data in transit).

Real-world use case: Standard command used to deploy/sync a codebase or website folder to a remote server.

```
rsync -avz --progress bigfile.tar.gz user@remotehost:/backup/
```

What it does: Same as above, plus a live progress bar showing transfer speed and percentage complete.

Real-world use case: Watching the transfer status of a large VM image or backup archive.

```
rsync -avzP ./data/ user@remotehost:/data/
```

What it does: -P is shorthand for --partial --progress: keeps partially transferred files if interrupted, and shows progress, so the transfer can resume later instead of restarting.

Real-world use case: Syncing large datasets over an unstable connection (e.g. homelab over Wi-Fi/VPN).

```
rsync -avz --delete ./src/ user@remotehost:/dest/
```

What it does: Mirrors the source exactly — deletes files on the destination that no longer exist in the source.

Real-world use case: Keeping a mirror/staging server identical to production, or cleaning up a backup destination after files were removed from source.

```
rsync -avz -e "ssh -p 2222" ./app/ user@remotehost:/opt/app/
```

What it does: The -e flag specifies the remote shell command, letting you pass custom SSH options like a non-default port or specific identity file.

Real-world use case: Syncing to a server with SSH running on a custom port or requiring a specific key.

```
rsync -avzn ./project/ user@remotehost:/var/www/project/
```

What it does: Dry run (-n / --dry-run) — shows exactly what WOULD be transferred/deleted without actually making changes.

Real-world use case: Safely previewing a --delete sync before running it for real, to avoid data loss.

```
rsync -avz --exclude='*.log' --exclude='node_modules/' ./app/ user@remotehost:/app/
```

What it does: Excludes specific files or directories (by pattern) from the sync.

Real-world use case: Deploying a Node.js/Python app while skipping log files, node_modules, or .git folders.

```
rsync -avz --include='*.jpg' --exclude='*' ./photos/ user@remotehost:/photos/
```

What it does: Combines include/exclude rules to sync only files matching a specific pattern.

Real-world use case: Backing up only image files from a mixed-content directory.

```
rsync -avz /source/ /destination/
```

What it does: Local-to-local sync (no remote host, no SSH) — still uses the same delta-transfer engine.

Real-world use case: Syncing files between two mounted drives, e.g. copying VM images between local disks in a homelab (ESXi datastore to backup NAS mount).

```
rsync -av --backup --backup-dir=/backups/$(date +%F) ./data/ user@remotehost:/data/
```

What it does: Before overwriting a file, moves the old version into a timestamped backup directory instead of discarding it.

Real-world use case: Syncing config/data while keeping versioned backups of anything that gets overwritten.

```
rsync -avz --bwlimit=1000 ./backup/ user@remotehost:/backup/
```

What it does: Limits bandwidth usage to the given KB/s during the sync.

Real-world use case: Running a large nightly backup job without saturating office/homelab bandwidth.

```
rsync -avz --delete-after ./site/ user@remotehost:/var/www/site/
```

What it does: Deletes extraneous destination files only after the transfer completes, rather than before/during — safer for zero-downtime deploys.

Real-world use case: Deploying a live website where the old version should stay served until the new files are fully in place.

```
rsync -avz -u ./project/ user@remotehost:/project/
```

What it does: -u (update) skips files that are newer on the destination than on the source.

Real-world use case: Two-way-ish syncing where you don't want to accidentally overwrite newer remote edits.

```
rsync -avz --link-dest=/backups/previous ./data/ /backups/$(date +%F)/
```

What it does: Creates hard links to unchanged files from a previous backup, so only changed files consume new disk space — the basis of incremental backup snapshots.

Real-world use case: Building rsync-based incremental backups (similar to Time Machine) on a home server.

Common rsync flags at a glance

Flag	Meaning
-a	Archive mode: recursive + preserves permissions, times, symlinks, ownership
-v	Verbose output
-z	Compress data during transfer
-P	= --partial --progress (resumable, shows progress)
--delete	Delete files on destination not present in source
-n	Dry run — preview only, no changes made
-e	Specify remote shell (e.g. custom SSH port/key)
--exclude / --include	Filter which files/folders are synced
--bwlimit	Cap transfer bandwidth
-u	Skip files newer on the destination

When rsync shines

- Repeated backups or deployments where only changed files should transfer (saves time and bandwidth).
- Mirroring servers, syncing large directory trees, or nightly cron backup jobs.
- Unreliable networks — transfers can resume instead of restarting from zero.
- Homelab scenarios: syncing VM images, Ansible playbook trees, or ISO repositories between hosts.

3. SFTP (SSH File Transfer Protocol)

SFTP opens an interactive, session-based file transfer shell over SSH. Unlike SCP/rsync (which are single-shot commands), SFTP lets you browse remote directories, list files, and transfer things back and forth within one live session — similar to old-school FTP, but encrypted. It's also usable as a subsystem for automated/scripted transfers.

Starting a Session

```
sftp user@remotehost
```

What it does: Opens an interactive SFTP session/shell connected to the remote host.

Real-world use case: The first step before browsing and moving files interactively, e.g. troubleshooting file layout on a client's server.

```
sftp -P 2222 user@remotehost
```

What it does: Connects using a custom SSH port.

Real-world use case: Connecting to a server where SSH is hardened to run on a non-default port.

```
sftp -i ~/.ssh/id_rsa_key user@remotehost
```

What it does: Connects using a specific private key file.

Real-world use case: Accessing a cloud host (EC2, DigitalOcean droplet) that requires key-based auth.

Commands Inside an SFTP Session

Command	What it does
<code>ls</code>	Lists files in the current remote directory.
<code>lls</code>	Lists files in the current LOCAL directory (note the leading 'l').
<code>cd /var/www/html</code>	Changes the current directory on the remote host.
<code>lcd /home/user/downloads</code>	Changes the current directory on the local machine.
<code>pwd</code>	Prints the current remote working directory.
<code>lpwd</code>	Prints the current local working directory.
<code>get file.txt</code>	Downloads a file from the remote host to the local machine.
<code>get -r remote_folder/</code>	Recursively downloads an entire remote directory.
<code>put file.txt</code>	Uploads a local file to the current remote directory.
<code>put -r local_folder/</code>	Recursively uploads an entire local directory.
<code>mkdir new_folder</code>	Creates a new directory on the remote host.
<code>rmdir old_folder</code>	Removes an (empty) directory on the remote host.
<code>rm file.txt</code>	Deletes a file on the remote host.
<code>rename old.txt new.txt</code>	Renames/moves a file on the remote host.
<code>chmod 755 script.sh</code>	Changes permissions on a remote file.
<code>df -h</code>	Shows disk space usage on the remote host (if supported by the server).
<code>exit / bye</code>	Closes the SFTP session.

Scripted / Batch SFTP

```
sftp -b batchfile.txt user@remotehost
```

What it does: Runs a batch file containing a sequence of SFTP commands non-interactively (-b).

Real-world use case: Automating a nightly file pickup/drop-off job in a shell script or cron job without manual interaction.

```
echo -e "cd /uploads\nput report.csv\nbye" | sftp user@remotehost
```

What it does: Pipes a sequence of commands directly into sftp instead of using a separate batch file.

Real-world use case: Quick one-off scripted upload inside a larger automation pipeline.

When SFTP shines

- Interactive exploration of a remote filesystem before deciding what to transfer.
- Environments where FTP-like browsing (list, navigate, rename, delete) is needed but plain FTP is not secure enough.
- Third-party or client file exchanges where a full shell login isn't offered, only SFTP access (common with shared hosting and managed file-transfer services).
- GUI clients (FileZilla, WinSCP, Cyberduck) — most connect using SFTP under the hood.

4. SCP vs RSYNC vs SFTP — Comparison

Aspect	SCP	RSYNC	SFTP
Transfer style	Full copy every time	Delta (only changed parts)	Full copy per file, interactive
Resume support	No	Yes (--partial / -P)	Partial, client-dependent
Speed on repeat syncs	Slow (always full copy)	Fast (only diffs move)	Slow (no diffing)
Interactive browsing	No	No	Yes (ls, cd, etc.)
Mirror/delete support	No	Yes (--delete)	Manual (rm, rmdir)
Bandwidth throttling	-l flag	--bwlimit flag	Client-dependent
Local-to-local sync	Yes (basic copy)	Yes, delta-based	No (remote sessions only)
Scripting/automation	Very easy, one-liner	Very easy, one-liner	Needs batch file (-b)
Best for	Quick single-file/folder copy	Backups, mirrors, large repeated syncs	Interactive/managed file exchange
Security	SSH encrypted	SSH encrypted (when used with -e ssh)	SSH encrypted
Availability by default	Almost universal on Linux	Usually needs rsync installed	Built into OpenSSH server

Advantages of Each, Head-to-Head

RSYNC over SCP

- Only transfers changed bytes, not the whole file again — massive time savings on repeated syncs.
- Can resume an interrupted transfer instead of starting over.
- Can delete stale files to keep a true mirror; SCP has no concept of this.
- Preserves far more metadata by default (permissions, ownership, symlinks) via -a.

SCP over RSYNC

- Simpler and installed everywhere by default — nothing extra to install on minimal/locked-down systems.
- Lower overhead for a genuine one-time, single-file copy where diffing adds no value.
- Slightly simpler syntax for very basic push/pull tasks in quick scripts.

SFTP over SCP/RSYNC

- Lets you browse, rename, delete, and inspect the remote filesystem interactively — neither SCP nor rsync offers a live shell.
- Works well for external/client file exchange where only SFTP access is granted (no full SSH shell).
- Natural fit for GUI file-transfer clients (FileZilla, WinSCP) used by non-CLI users.

SCP/RSYNC over SFTP

- Both are far faster for scripted, non-interactive bulk transfers — no session overhead.
- rsync in particular is dramatically more efficient for large, recurring, mostly-unchanged datasets.
- Better suited to cron jobs, CI/CD pipelines, and backup automation where no human is present.

Quick Decision Guide

Scenario	Best Tool
Copy one file to a server, one time	SCP

Nightly backup of a large, mostly-unchanged folder	RSYNC
Mirror a website/staging folder to match production exactly	RSYNC --delete
Browse a client's server and manually manage a few files	SFTP
Deploy code in a CI/CD pipeline step	RSYNC (or SCP for very small payloads)
Transfer over a flaky/unstable network connection	RSYNC (-P for resume)
Automated scripted upload with no rsync installed	SCP or scripted SFTP (-b)
Homelab: sync ISO/VM images between two hosts	RSYNC

Note: SCP is being deprecated in favor of SFTP-based transfer within newer OpenSSH releases due to protocol weaknesses in the legacy SCP protocol. Many distros now alias `scp` to use the SFTP protocol internally under the hood, though the command-line usage stays the same.